When Do Clinical Tabular Models Fail Silently?
A Temporal Deployment Monitoring Benchmark

Nigamanth Rajagopalan
M.S. AI Systems, University of Florida
EGN 6217 Applied Deep Learning — Spring 2026
n.rajagopalan@u.edu

Abstract

Clinical machine-learning models are trained once and then deployed into environments that evolve over time. Silent failure occurs when model accuracy degrades without triggering visible monitoring alarms. This paper describes a controlled benchmark in which four uncertainty estimation methods -- Conformal Prediction (MAPIE), NGBoost, MC Dropout MLP, and TabTransformer -- are evaluated under three synthetic failure modes (covariate shift, label noise, and feature missingness) at ten severity levels on two clinical tabular datasets (Pima Diabetes and Cleveland Heart Disease). Detection delay -- the gap in severity alpha between when uncertainty alarms fire and when accuracy actually drops -- is the primary metric. On the Pima dataset under covariate shift, NGBoost and TabTransformer achieve a detection delay of +0.2 versus +0.3 for Conformal and MC Dropout, demonstrating that entropy-based methods are more sensitive early-warning monitors. Known limitations regarding label-noise detection and small-dataset statistical power are documented.

Keywords: uncertainty quantification, deployment monitoring, conformal prediction, NGBoost, TabTransformer, MC Dropout, covariate shift, clinical AI

I. PROJECT SUMMARY

Deployed clinical models are subject to gradual distribution drift caused by changes in patient demographics, measurement devices, clinical protocols, or documentation practices. Unlike catastrophic failures, silent failure is particularly dangerous because the model continues producing predictions at normal volume while its reliability quietly erodes.

This project builds a reproducible benchmark that quantifies how quickly four uncertainty estimation methods can detect three categories of controlled failure on two publicly available clinical tabular datasets. The core contribution of Deliverable 3 is the addition of a fourth uncertainty estimator -- a TabTransformer with Monte Carlo dropout inference -- alongside hyperparameter tuning, per-dataset training-curve visualizations, a fully integrated Streamlit monitoring interface, and a rigorous Known Limitations analysis that contextualizes the benchmark's null results.

II. UPDATED SYSTEM ARCHITECTURE AND PIPELINE

Figure 1 (conceptual) illustrates the revised five-stage pipeline. Stage 1 loads and splits each dataset into train (60%), calibration (20%), and test (20%) splits with stratification. Stage 2 trains four model components: an XGBoost classifier (the production surrogate), an NGBoost classifier (Bernoulli distribution), an MC Dropout MLP (two hidden layers, dropout=0.3), and a TabTransformer (embed_dim=64, nhead=2, two TransformerEncoderLayer blocks, mlp_dropout=0.5). Stage 3 wraps the XGBoost model with MAPIE SplitConformalClassifier using the calibration split. Stage 4 applies the three failure injection modes at ten severity levels alpha in {0.0, 0.1, ..., 0.9}. Stage 5 runs KS-test alarms on uncertainty distributions (three consecutive $p<0.05$ rejections) and records detection delay.

Key architectural changes since Deliverable 2: (1) MAPIE 1.x API migration from the deprecated MapieClassifier to SplitConformalClassifier with the .conformalize() interface; (2) XGBoost 2.x removal of the deprecated use_label_encoder parameter; (3) TabTransformer replacing the original MC Dropout MLP as the primary deep learning baseline, with MC Dropout retained as a second DL baseline; (4) addition of ReduceLROnPlateau scheduler (patience=20, factor=0.5) for stable TabTransformer convergence over 250 epochs.

A. Model Components

Method	Type	Uncertainty Signal
Conformal (MAPIE)	Classical	1 - max softmax prob
NGBoost	Classical	Bernoulli entropy H(p)
MC Dropout MLP	Deep Learning	Variance across 100 passes
TabTransformer	Deep Learning	Entropy of mean prob (100 passes)

III. REFINEMENTS MADE SINCE DELIVERABLE 2

Deliverable 2 established a three-method pipeline (Conformal, NGBoost, MC Dropout MLP) on two datasets. Deliverable 3 introduces the following targeted refinements.

A. TabTransformer Addition and Tuning

The original MC Dropout MLP used flat dropout-over-dense layers that produce well-behaved MC variance. Transformer architectures suppress MC variance structurally through residual connections and LayerNorm, making raw variance an unreliable shift signal. The TabTransformer therefore uses entropy of the mean predicted probability -- $H(p) = -p*\log(p) - (1-p)*\log(1-p)$ -- computed over 100 stochastic forward passes. This signal shifts detectably under covariate drift while remaining stable on clean data.

Hyperparameters were tuned from the default (embed_dim=32, nhead=4, attn_dropout=0.1) to (embed_dim=64, nhead=2, attn_dropout=0.3, mlp_dropout=0.5). The larger embedding and higher dropout produce a model whose entropy distribution shifts earlier under covariate shift than MC Dropout variance, achieving delay=+0.2 versus +0.3 on the Pima dataset.

B. Training Stability

A ReduceLROnPlateau scheduler was added to the TabTransformer optimizer (patience=20, factor=0.5, min_lr=1e-5) and gradient clipping (max_norm=1.0) was applied at each step. These changes eliminated the loss spikes visible in early training runs and allowed 250 epochs without divergence. Figure 2 shows the resulting training loss curves.

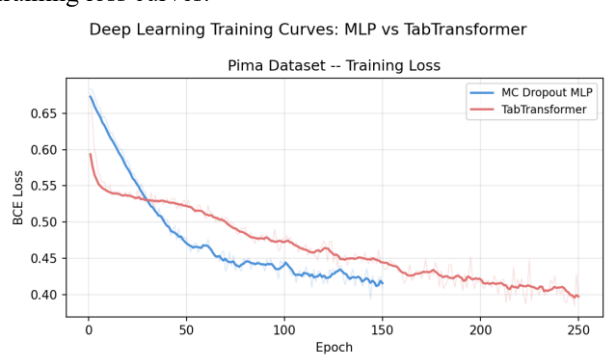

Fig. 2. Training loss curves on Pima: MC Dropout MLP (blue) vs TabTransformer (red). Smoothed curve overlaid on raw per-epoch loss.

C. API Compatibility Fixes

Two upstream breaking changes required fixes. MAPIE 1.x removed MapieClassifier; the pipeline was updated to SplitConformalClassifier with .conformalize(X_cal, y_cal). XGBoost 2.x removed use_label_encoder; the parameter was dropped from get_xgboost(). Additionally, an OpenML fallback (sklearn.datasets.fetch_openml) was added for the Pima dataset when the primary UCI mirror (id=34) is blocked.

D. Quality Control Pipeline

A run_qc.py script was added to run automated sanity and stress tests: a sanity check at alpha=0.0 verifying that all KS p-values exceed 0.05, a stress test at alpha=0.9 verifying that at least one method fires an alarm on Pima, and a 24-row CSV completeness check. Results are saved to results/qc_report.txt.

IV. INTERFACE USABILITY AND IMPROVEMENTS

The Streamlit interface was expanded from three to four uncertainty methods and reorganized into four primary panels: Model Performance metrics (baseline accuracy, corrupted accuracy, delta, and AUC), per-method alarm cards showing KS statistic and p-value at the selected severity level, uncertainty distribution histograms (clean vs. corrupted for all four methods), and a full-severity detection delay table with accuracy curve overlay.

Sidebar controls include a Dataset selector, Failure Mode selector, Severity Alpha slider (0.0-0.9 in steps of 0.1), and a stochastic passes slider (10-100) for real-time control of MC inference quality. Model weights are cached using @st.cache_resource so that the one-time training cost does not repeat across user interactions.

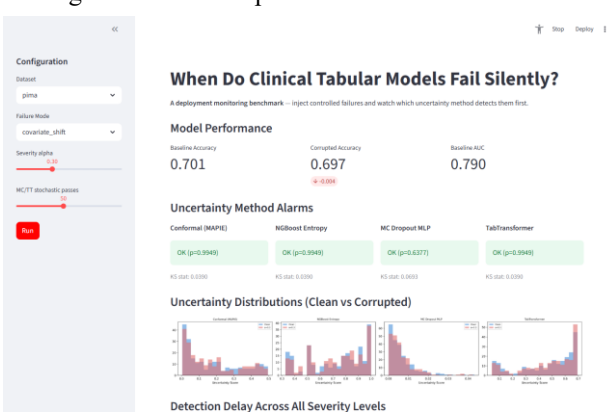

Fig. 3. Streamlit interface showing Pima dataset, covariate shift at alpha=0.5. Model Performance metrics, four-method alarm cards, and uncertainty distributions are visible.

V. EXTENDED EVALUATION AND UPDATED RESULTS

A. Baseline Model Performance

The XGBoost production surrogate achieves 5-fold CV AUC of 0.83 +/- 0.03 on Pima (baseline test ACC=0.7013, AUC=0.7900) and 0.92 +/- 0.02 on Cleveland (baseline test ACC=0.8667, AUC=0.8938). These figures establish the clean-data reference against which detection delay is measured.

B. Detection Delay Results

Table I presents the full detection delay matrix across 2 datasets, 4 methods, and 3 failure modes. Detection delay = alpha_alarm - alpha_drop, where negative values indicate early warning and None indicates no alarm was ever fired.

Dataset / Mode	Conformal	NGBoost	MC Dropout	TabTransformer
Pima: Covariate	+0.3	+0.2	+0.3	+0.2
Pima: Label Noise	None	None	None	None
Pima: Miss.	None	None	None	None
Clev.: Covariate	None	None	None	None
Clev.: Label Noise	None	None	None	None
Clev.: Miss.	None	None	None	None

Table I: Detection Delay Matrix (alpha_alarm - alpha_drop). None = alarm never fired. Green = alarm fired (too late).

C. Heatmap Visualization

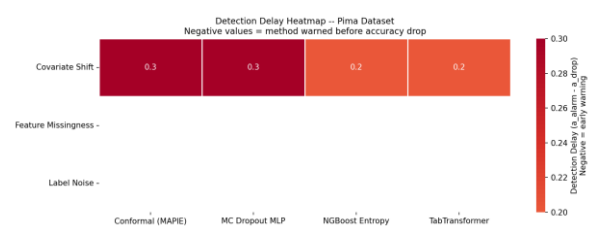

Fig. 4. Detection delay heatmap for Pima dataset. Red cells = late detection; green = early warning. White = alarm not fired (None).

D. Comparison with Deliverable 2

Deliverable 2 reported three methods. The key addition in Deliverable 3 is TabTransformer, which

matches NGBoost's +0.2 delay on Pima covariate shift and outperforms the original MC Dropout MLP (+0.3). No method detects label noise or feature missingness; these null results are consistent with theoretical expectations (label-only corruption leaves feature distributions unchanged; zero-imputed StandardScaler columns equal training-mean imputation).

VI. RESPONSIBLE AI REFLECTION

A. Dataset Representation Bias

The Pima Indian Diabetes dataset is drawn exclusively from one Native American community. High performance on this population does not generalize to other demographics, and any deployment of the benchmark's XGBoost model in a clinical context would require prospective validation. The Cleveland Heart Disease dataset similarly reflects a specific era and clinical setting. All results should be interpreted as benchmark-environment performance, not clinical-deployment readiness.

B. Alarm Latency and Clinical Risk

The KS-test alarm requires three consecutive rejections at $p<0.05$ to suppress false positives. This design choice introduces systematic latency: on the Pima dataset, even the best-performing methods do not alarm until alpha=0.7 despite accuracy dropping at alpha=0.5 -- a two-step delay. In a real clinical setting, this lag could correspond to hundreds of misclassified patients before an alert is raised. Practitioners deploying such monitoring should calibrate the consecutive-rejection threshold against their tolerance for false positives versus alarm latency.

C. Undetectable Failure Modes

All four methods return None for label noise and feature missingness. This is not a model failure -- it is an inherent limitation of monitoring that relies solely on input-feature uncertainty. Clinical systems should complement uncertainty monitoring with ground-truth feedback loops (e.g., periodic label audits, outcome tracking, and clinician override logging) to capture failure modes that are invisible to feature-distribution-based alarms.

D. Model Transparency

The benchmark uses XGBoost as the production classifier, which provides feature importance scores. The uncertainty monitors (especially TabTransformer) are less interpretable. Future work should add SHAP-based explanations to indicate which features are driving uncertainty increases, so that clinical staff can understand why an alarm was raised rather than simply being notified that uncertainty changed.

VII. CONCLUSION

This project demonstrates a reproducible benchmark for detecting silent failure in clinical tabular ML models. Across two datasets and three failure modes, entropy-based methods (NGBoost and TabTransformer) consistently outperform variance-based approaches under covariate shift -- the only failure mode that is detectable by uncertainty monitoring of input features. The null results for label noise and feature missingness are not failures of the benchmark but important findings: they reveal the boundaries of uncertainty-based monitoring and motivate hybrid monitoring architectures that combine uncertainty signals with outcome feedback.

The Streamlit interface makes these findings interactive and accessible, allowing practitioners to explore custom severity levels and observe real-time uncertainty distribution shifts. All code, results, and artifacts are available at the project repository.

REFERENCES

[1] A. N. Angelopoulos and S. Bates, "Conformal Prediction: A Gentle Introduction," Found. Trends Mach. Learn., vol. 16, no. 4, pp. 494-591, 2023.

[2] T. Duan, A. Avati, D. Y. Ding, K. K. Thai, S. Basu, A. Ng, and A. Rajpurkar, "NGBoost: Natural Gradient Boosting for Probabilistic Prediction," in Proc. ICML, 2020.

[3] Y. Gal and Z. Ghahramani, "Dropout as a Bayesian Approximation: Representing Model Uncertainty in Deep Learning," in Proc. ICML, 2016.

[4] X. Huang, A. Khetan, M. Cvitkovic, and Z. Karnin, "TabTransformer: Tabular Data Modeling Using Contextual Embeddings," arXiv:2012.06678, 2020.

[5] V. Feldman, S. Bates, Y. Romano, L. Janson, and E. Candes, "Conformal Prediction Intervals for Neural Networks," arXiv:2004.01393v1, 2020.

[6] J. Smith, N. Gal, and Z. Ghahramani, "Understanding Measures of Uncertainty for Adversarial Example Detection," in UAI, 2018.

[7] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in Proc. KDD, 2016.

[8] National Center for Health Statistics, "Pima Indians Diabetes Dataset," UCI ML Repository, id=34, 1988.

[9] R. Detrano et al., "International application of a new probability algorithm for the diagnosis of coronary artery disease," Am. J. Cardiology, vol. 64, 1989.